Retrieval Augmented Generation

Why RAG Exists (The Problem First)

LLMs have two fundamental limitations:

```
1. Knowledge cutoff     → doesn't know anything after training date
2. No private data      → never saw your company's internal documents
```

The naive solution is fine-tuning — train the model on your data. But fine-tuning is:

- Expensive
- Slow
- Still doesn't update in real-time
- Model can still hallucinate "memorized" facts incorrectly

RAG solves this differently. Instead of **baking knowledge ,you** fetch relevant knowledge at query time** and hand it to the model as context.

The model is simple:

```
Without RAG:  Question → LLM (guesses from memory) → Answer
With RAG:     Question → Fetch relevant docs → LLM (reads docs) → Grounded Answer – factual truth
```

The LLM stops being an encyclopedia and becomes a **reasoning engine over retrieved context.**

The Two Phases of RAG

Every RAG system has exactly two phases :

Phase 1 — Indexing () runs once)

```
Raw Documents
     ↓
Document Loader        → creates Document objects
     ↓
Text Splitter          → breaks into chunks
     ↓
Embedding Model        → converts each chunk to a vector
```

```
     ↓
Vector DB              → stores vectors + metadata + original text
```

This happens **before any user query.** It's a one-time pipeline (re-run only when documents change). The Vector DB is your pre-built knowledge index.

Phase 2 — Retrieval + Generation (runs every query)

```
User Query
     ↓
Same Embedding Model  → converts query to query vector
     ↓
Vector DB ANN Search  → finds Top-K most similar chunk vectors
     ↓
Fetch chunk text      → get the actual text of those chunks
     ↓
Build prompt          → "Answer this question using these chunks: ..."
     ↓
LLM                   → reads chunks, generates grounded answer
     ↓
Response to User
```

What the LLM Actually Sees

The LLM never directly searches the vector DB. It just receives a prompt that looks like this:

```
System: You are a helpful assistant. Answer only using the
        context provided below. If the answer is not in the
        context, say "I don't know."

Context:
[Chunk 1]: "Ibuprofen is a nonsteroidal anti-inflammatory drug
            used to reduce fever and treat pain..."
[Chunk 2]: "Common side effects of NSAIDs include stomach upset,
            nausea, and in rare cases kidney issues..."
[Chunk 3]: "Ibuprofen should not be taken on an empty stomach..."

User Question: What are the side effects of ibuprofen?
```

The LLM just reads this like a comprehension exercise. **Retrieval finds the pages. The LLM reads them.**

The Three Things RAG Requires to Work Well

```
1. Good Chunking        → right size, right boundaries
2. Good Embedding Model → captures semantic meaning accurately
3. Good Prompt Design   → LLM knows how to use the retrieved context
```

If any one of these is weak, the whole system underperforms — even with a powerful LLM.

Q: Why can't we just put the entire document in the LLM's context window?**

You can, for small documents. This is called **"stuffing"** and is valid for small use cases. But it breaks at scale because:

- LLMs have context limits (even 128K tokens fills up fast with large corpora)
- Cost scales linearly with tokens — stuffing 100 docs per query is expensive
- LLM attention degrades on very long contexts ("lost in the middle" problem — research shows LLMs recall beginning and end of context well, but poorly recall middle sections)
- RAG retrieves only what's relevant — cleaner, cheaper, more precise

Q: Why not just fine-tune the LLM on our documents instead?**

Fine-tuning teaches the model **how to behave and what style to use.**

It does not reliably teach it to **recall specific facts.**

Models fine-tuned on facts still hallucinate those facts under paraphrase.

RAG gives the model the fact at query time — no memorization needed. In production, fine-tuning and RAG are often combined — fine-tune for tone/domain behavior, RAG for factual grounding.

Q: What does the Vector DB actually store — the vector or the text?**

Both. The vector is used for similarity search.

The original chunk text is stored as a payload/metadata alongside it.

When retrieval returns Top-K results, you fetch the text of those chunks — that text goes into the LLM prompt. The LLM never sees raw vectors.

Q: Does the LLM know it's receiving retrieved context? Does it matter?**

The LLM has no inherent awareness — it just processes the prompt. What matters is your **system prompt instructions** telling the LLM to use the provided context, stay grounded in it, and say "I don't know" if the answer isn't there.

Without these instructions, the LLM may ignore context and answer from its own training weights anyway.

Q: What if the right answer spans multiple chunks?**

This is one of the hardest problems in RAG, called **multi-hop retrieval.** A question like "Compare the revenue of our Q1 and Q3 reports" needs information from two separate chunks that may not be similar to each other or to the query. Solutions:

- Retrieve more chunks (higher K)
- Use a knowledge graph layer alongside vector search
- Use agentic RAG where the LLM makes multiple retrieval calls

Q: What is Top-K and how do we decide K?**

Top-K is how many chunks you retrieve per query. Typical values are 3-10.

Too low → you miss relevant context.

Too high → you flood the LLM prompt with noise, increasing cost and confusing the model. K is a hyperparameter you tune based on eval metrics .

What if the query is ambiguous or poorly worded or you asked questions in a lingo ?

You have RAG system built on a **company's HR policy documents.** The documents contain detailed information about leave policies, maternity leave, sick leave, etc.

A user asks:

```
"what about leave?"
```

This is ambiguous. Vague. Poorly worded. Let's see what happens and how each technique fixes it.

What Happens Without Any Fix

The embedding model encodes `"what about leave?"` as a vector.

This vector sits in a **very generic region** of the vector space — because the sentence carries almost no semantic signal. "Leave" could mean:

- Leaving the company (resignation)
- Annual leave / vacation
- Sick leave
- Maternity leave
- Unpaid leave

The vector DB does ANN search and returns the **closest chunks to that vague vector** — which might be loosely related to HR, or might be completely off. Retrieval quality is poor. The LLM gets bad context. Answer is bad or generic.

Solution 1 — Query Rewriting

Idea: Before embedding, use an LLM to rewrite the vague query into a cleaner, more specific, information-seeking question.

How it works:

```
Original query:  "what about leave?"
        ↓
Send to LLM with prompt:
"Rewrite this query into a clear, specific question
 that someone might search for in an HR document:
 Query: 'what about leave?'"
        ↓
LLM rewrites it to:
"What are the types of employee leave available and
 what is the policy for each?"
        ↓
NOW embed this rewritten query → much richer vector
        ↓
Vector DB retrieves relevant leave policy chunks
```

Why it works: The rewritten query has actual semantic content — "types of employee leave", "policy for each." The embedding model now has real meaning to encode. The resulting vector lands in a much more specific, relevant region of the vector space.

Trade-off: One extra LLM call before retrieval. Adds ~200-400ms latency.

Solution 2 — Query Expansion

Idea: Don't rewrite into one better query — generate **multiple different phrasings** of the query, retrieve for all of them, then merge the results.

How it works:

```
Original query: "what about leave?"
        ↓
Send to LLM:
"Generate 4 different ways an employee might ask
 about leave policy in an HR system"
        ↓
LLM generates:
  1. "How many sick days am I entitled to per year?"
  2. "What is the maternity leave policy?"
  3. "How do I apply for annual leave?"
  4. "Can I carry forward unused leave to next year?"
        ↓
Embed ALL 4 queries → 4 separate query vectors
        ↓
Run 4 ANN searches in Vector DB → 4 sets of Top-K chunks
        ↓
Merge all results, deduplicate, rerank by relevance
        ↓
Send best chunks to LLM
```

Why it works: You're casting a wide net. Even if you don't know exactly what the user meant, you cover multiple possible intents. The union of retrieved chunks is far more comprehensive.

Trade-off: Multiple vector DB searches instead of one. More retrieved chunks means larger prompt and more LLM tokens. But for ambiguous queries, recall improves dramatically.

Solution 3 — HyDE (Hypothetical Document Embedding)

This one is counterintuitive and clever. Instead of trying to improve the query, you ask the LLM to **hallucinate a fake answer** — as if the perfect document already existed — and embed that fake answer as your search vector.

How it works:

```
Original query: "what about leave?"
        ↓
Send to LLM:
"Write a short paragraph that would be found in an
 HR policy document that directly answers this
 question: 'what about leave?'"
```

```
        ↓
LLM generates a HYPOTHETICAL document passage:
"Employees are entitled to 20 days of annual leave
 per year. Sick leave is capped at 10 days with a
 medical certificate required after 3 consecutive
 days. Maternity leave is 26 weeks fully paid.
 Leave applications must be submitted via the HR
 portal at least 7 days in advance..."
        ↓
Embed THIS hypothetical passage → very rich vector
(not the original query — the fake answer)
        ↓
Use this vector to search the Vector DB
        ↓
Real document chunks that look like this passage
get retrieved
```

Why it works: This is the key insight —

```
Query vectors and document vectors live in the same space.
A hypothetical document passage LOOKS like a real document passage.
So it naturally lands near real document chunks in vector space.
A short vague query does NOT look like a document — it lands far away.
```

The fake answer essentially acts as a **semantic bridge** between the vague query and the actual document space.

Trade-off: The LLM might hallucinate irrelevant content in the hypothetical passage, which could steer retrieval in the wrong direction. Works best when the query has at least some domain signal.

Original Query: "what about leave?"

Query Rewriting: "What are the types of employee leave
and the policy for each?"
→ 1 clean query, 1 search

Query Expansion: 4 specific questions covering different
possible intents
→ 4 searches, merged results, wider net

HyDE: A fake HR document paragraph about leave
→ 1 search, but from document-space
instead of query-space

- **Query Rewriting** → *Clean up the question before searching*
- **Query Expansion** → *Ask the question multiple ways, take the best from all*
- **HyDE** → *Imagine the answer, search for documents that look like it*

Checking Recall@k and precision@k

Recall@K

```
"Out of all the queries I tested, in what percentage
 of them did the correct answer appear somewhere
 in the Top-K retrieved chunks?"
```

That's it. Nothing more complex than that.

Say you have an HR policy RAG system. You run **10 test queries** where you already know which chunk contains the correct answer.

You set **K = 5** — meaning the Vector DB returns the Top 5 most similar chunks per query.

```
Query 1:  "What is the notice period?"
          Top 5 retrieved chunks → correct chunk is #2  PASSED

Query 2:  "How many sick days per year?"
          Top 5 retrieved chunks → correct chunk is #1  PASSED

Query 3:  "What is the maternity leave policy?"
          Top 5 retrieved chunks → correct chunk is #5  FAILED

Query 4:  "Can I work remotely?"
          Top 5 retrieved chunks → correct chunk NOT in top 5  FAILED

Query 5:  "What is the travel reimbursement limit?"
          Top 5 retrieved chunks → correct chunk is #3  PASSED

Query 6:  "How do I raise a grievance?"
          Top 5 retrieved chunks → correct chunk NOT in top 5  FAILED

Query 7:  "What is the probation period?"
          Top 5 retrieved chunks → correct chunk is #4  PASSED

Query 8:  "Are there health insurance benefits?"
          Top 5 retrieved chunks → correct chunk is #1  PASSED

Query 9:  "What happens if I am late?"
```

```
          Top 5 retrieved chunks → correct chunk NOT in top 5  FAILED

Query 10: "How do I apply for leave?"
          Top 5 retrieved chunks → correct chunk is #2  PASSED
```

```
Hits    = 7  (queries where correct chunk appeared in Top 5)
Misses  = 3  (queries where correct chunk was NOT in Top 5)
Total   = 10

Recall@5 = 7/10 = 70%
```

K is just how many chunks you allow the retriever to return per query.

```
Recall@1  → Did the correct chunk come back as the #1 result?
Recall@3  → Did the correct chunk appear anywhere in Top 3?
Recall@5  → Did the correct chunk appear anywhere in Top 5?
Recall@10 → Did the correct chunk appear anywhere in Top 10?
```

Higher K → easier to get a higher Recall score, but you're retrieving more chunks → more noise into LLM prompt.

Recall@K does **not** care about position within the top K.

```
Correct chunk at position 1  .... counts as a hit
Correct chunk at position 5    .... also counts as a hit
Correct chunk at position 6    ...... miss (if K=5)
```

It only asks — **is it in the list, yes or no?**

This is why it is called Recall and not Precision. You are measuring **did we find it at all** — not **did we find it first.**

Why It Matters for RAG Specifically

```
If Recall@5 = 70%

It means 30% of the time, the correct chunk never
even reaches the LLM.

The LLM is working with wrong context 30% of the time.
```

```
No matter how good your LLM is — if the right chunk
is not retrieved, the answer will be wrong or hallucinated.
```

This is why Recall@K is the **first metric** you fix in a RAG system. Everything downstream depends on it.

```
Retrieval fails → LLM gets wrong context → Wrong answer
                                            (and you may never know)
```

Recall@K vs Precision@K

```
Recall@K:    Of all my test queries, how often was the
             right chunk in the top K?
             → Am I finding the right thing at all?

Precision@K: Of the K chunks I retrieved, how many
             were actually relevant?
             → Am I retrieving junk along with the right thing?
```

For RAG, **Recall@K comes first.** If the right chunk is not retrieved, precision does not matter. Once recall is solid, you tune precision — usually with a reranker.